

Particle Filter SLAM · 零基础工程报告

ESE 650 HW3 Problem 2 (KITTI 数据集)

Student ID: 18330723

2026 年 4 月 24 日

阅读说明：本报告假设你**没学过粒子滤波、没听过 SLAM、只会 numpy**。我会从 KITTI 这些 .bin 文件里到底装了什么讲起，一步步走到最终的地图和轨迹。每个工程决策都会问“为什么”，每段代码会说明它在干什么、为什么这样写。

目录

1 要解决的真实问题	2
1.1 本题的具体设定	2
2 先看数据：KITTI 到底给了什么	3
2.1 文件夹结构	3
2.2 LiDAR .bin 文件的格式	3
2.3 Poses 文件：车的轨迹	3
2.4 Calibration 文件	3
2.5 为什么需要 3 个坐标系？	4
3 为什么不能只用 odometry 或只用 LiDAR？	5
3.1 方案 A：只用 odometry 呢？	5
3.2 方案 B：只用 LiDAR (scan matching) 呢？	5
3.3 方案 C：两者融合	5
4 粒子滤波的基本思想（先脱离 SLAM 看它）	6
4.1 核心想法：用一群点近似一个分布	6
4.2 一个粒子是什么	6
4.3 粒子滤波的三大动作	6
5 数据结构 1：粒子集	7
6 数据结构 2：占据栅格地图	8
6.1 地图的形状	8
6.2 Log-odds：把贝叶斯更新变成加法	8
6.3 从世界坐标到地图格子	9

7 预测步：粒子怎么往前走？	10
7.1 从 pose 差分得到”控制信号”	10
7.2 为什么要用 smart_minus_2d? 直接做减法不行吗？	10
7.3 SE(2) 上的加法和减法	10
7.4 最终 predict 步	11
8 观测步：LiDAR 怎么更新粒子权重？	12
8.1 核心思路：一致性评分	12
8.2 LiDAR 投影链	12
8.3 清洗点云	12
8.4 计算观测 log-likelihood	13
8.5 log 域权重更新	13
9 更新地图：用哪个粒子？	14
10 重采样：何时、为什么	15
10.1 粒子退化问题	15
10.2 N_eff: 有效粒子数	15
10.3 Stratified Resampling	15
11 把所有步骤连起来：主循环	17
11.1 流程总结图	17
12 “毛刺”问题——轨迹为什么不光滑	18
12.1 现象	18
12.2 6 个根本原因	18
12.3 四种缓解方法	18
13 验证：怎么判断 SLAM 跑对了？	19
14 口试速查·常见陷阱	20
15 考题：按学习路径排序	21

1 要解决的真实问题

想象你是一辆自动驾驶车

你在一个不认识的城市里开，车头顶着一个 LiDAR（激光雷达），每隔 0.1 秒往周围所有方向发射激光，收集 10000 个点云点，告诉你：“这些方向上有障碍，距离是多少米”。

你想做两件事：

1. **定位** (Localization)：我现在在哪？朝哪开？
2. **建图** (Mapping)：这个城市的地图长啥样？

同时做这两件事就叫 **SLAM** (Simultaneous Localization And Mapping) 。

为什么 SLAM 是个”鸡生蛋”问题？

- 要**定位**：你得知道周围长啥样（地图），把你眼前看到的和地图里的对上。
- 要**建图**：你得知道自己在哪，才能把眼前看到的“贴”到地图正确位置。

两件事互相依赖。你只有同时做，而且**迭代**地做，才能解开。

1.1 本题的具体设定

车：KITTI 数据集里的一辆车，在 10 Hz 的 LiDAR 扫描中开过一段街道。

已知：

- 每帧 LiDAR 点云（~ 10000 个 3D 点）。
- 每帧车的**粗略 pose**（从 GPS/IMU 融合来，有噪声，不是完美真值）。
- 相机和 LiDAR 之间的坐标变换矩阵（calibration）。

要产出：

- 一张 2D 占据栅格地图（告诉你哪些地方有墙/树/车）。
- 一条更精细的车辆轨迹（比 odom 更准）。

2 先看数据：KITTI 到底给了什么

2.1 文件夹结构

KITTI 数据布局

```
KITTI/
  odometry/02/velodyne/    # LiDAR .bin 文件，每帧一个
    000000.bin             # 10K+ 个点，每点 4 float32
    000001.bin
    ... (4660 帧)
  poses/02.txt             # 每行一个 3x4 矩阵（车的 pose）
  calib/02/calib.txt       # LiDAR->camera 变换矩阵 Tr
```

2.2 LiDAR .bin 文件的格式

每个 .bin 文件是一段连续 float32 二进制数据：

$$\underbrace{[x_1, y_1, z_1, i_1]}_{\text{点 1}}, \underbrace{[x_2, y_2, z_2, i_2]}_{\text{点 2}}, \dots]$$

用 numpy 读：

```
points = np.fromfile(bin_path, dtype=np.float32).reshape(-1, 4)
# points.shape = (N, 4), 例如 (123456, 4)
# 列 0,1,2: x, y, z 坐标（米，LiDAR 本地坐标系 = "velodyne frame"）
# 列 3: intensity（反射强度，0 ~ 1 左右）
```

物理意义：

- (x, y, z) : 障碍物相对 LiDAR 的 3D 位置.
- intensity: 反射强度，越高说明激光击中了反射率好的东西（金属、标志）；越低可能是空气/噪声.

2.3 Poses 文件：车的轨迹

poses/02.txt 每行是一个 3×4 矩阵 $[R | t]$ （12 个数），表示车在世界坐标系下的姿态.

```
T = np.array([float(x) for x in line.split()]).reshape(3, 4)
# T = [R | t], R is 3x3 rotation, t is 3x1 translation
# Rotation from car frame to world frame
```

2.4 Calibration 文件

Tr: 一个 3×4 矩阵，把点从 velodyne frame 变到 camera frame:

$$\mathbf{p}_{\text{cam}} = T_r \cdot [\mathbf{p}_{\text{velo}}; 1]$$

这是因为 KITTI 的 pose 是以 camera frame 为基准的，而 LiDAR 在车上的位置不一样.

2.5 为什么需要 3 个坐标系?

坐标系链

$$\text{velodyne} \xrightarrow{T_r} \text{camera} \xrightarrow{\text{car pose}} \text{world}$$

- **velodyne frame:** LiDAR 自己的坐标系, 点云的原始坐标.
- **camera frame:** 车上 camera 的坐标系, pose 以它为基准.
- **world frame:** 全局固定坐标系, 地图在这里建.

建图前必须把所有点一路推到 world frame 才有意义.

3 为什么不能只用 odometry 或只用 LiDAR?

3.1 方案 A: 只用 odometry 呢?

odometry 给了每帧的 pose, 按理说直接把 LiDAR 点云全投影到 world 就能建图。

问题:

- odom 本身有噪声 (GPS 可能偏几米, IMU 积分会漂)。
- 错误会累积: 开几百米后位置可能偏出 5 米。
- 结果: 地图上墙会变成“双层” (ghost walls), 无法使用。

3.2 方案 B: 只用 LiDAR (scan matching) 呢?

每一帧和上一帧的 LiDAR 做配准 (比如 ICP 算法), 估出相对运动。

问题:

- 两帧间初值不好会陷入局部最优。
- 没有全局先验 → 长距离累积误差更严重。
- 自相似环境 (长直走廊、停车场) 会完全失败 (多模)。

3.3 方案 C: 两者融合

odom 提供运动先验 (车大致去哪了), LiDAR 提供观测纠正 (实际环境长啥样)。

但这里有个关键问题: 后验分布可能多模 —— 比如在十字路口, 两个粒子哪个是真位置都说得通。用 EKF 的单高斯模型处理不了多模。

所以要用粒子滤波

粒子滤波 (Particle Filter, PF): 用一群样本点表示后验分布, 每个样本点是一个“这个车可能在哪”的假设。

- 可以表示任意分布 (不用高斯)。
- 观测模型不用可微。
- 天然支持多模。

代价: 粒子数有限 (本题 $N = 50$) → Monte Carlo 方差。

4 粒子滤波的基本思想（先脱离 SLAM 看它）

4.1 核心想法：用一群点近似一个分布

假设你想表示“我大概在某个位置附近”这个信念。传统方法：用一个高斯 $\mathcal{N}(\mu, \Sigma)$ 。

粒子滤波方法：撒 N 个点（每个叫一个“粒子”），每个点是位置的一个假设，附带一个权重 w_i 表示“我觉得这个假设有多可信”。

$$p(x) \approx \sum_{i=1}^N w_i \delta(x - x^{(i)})$$

4.2 一个粒子是什么

在本题中，一个粒子 = 一个车的位姿假设 $[x, z, \theta]$ ——位置 (x, z) 加上朝向 θ 。

为什么只有 3 维（不是 6 维）？因为 KITTI 车在平面上开： y 方向几乎不变（海拔），俯仰和翻滚几乎为 0，只剩 x/z 平移 + 绕 y 轴的 yaw。

粒子在 numpy 里的表示

```
s.p = np.zeros((3, s.n)) # (3, N) 每列一个粒子 [x; z; theta]
s.w = np.ones(s.n) / s.n # (N,) 权重，一开始均匀分布
```

4.3 粒子滤波的三大动作

1. **预测 (Predict)**：每个粒子按运动模型往前走一步（加噪声）。
2. **更新 (Update)**：每个粒子按观测似然重新加权（一致的观测 → 权重大）。
3. **重采样 (Resample)**：杀掉权重小的、复制权重大的，保持总数 N 。

这是所有 PF 的通用骨架，后面我们一个个填具体的模型。

5 数据结构 1: 粒子集

本题的 `slam_t` 类的粒子相关成员:

粒子初始化

```
def init_particles(s, n=100, p=None, w=None):  
    s.n = n  
    s.p = deepcopy(p) if p is not None else np.zeros((3, s.n))  
    s.w = deepcopy(w) if w is not None else np.ones(n) / n
```

本题实际初始化:

- $N = 50$ 个粒子.
- 全部初始化在第一帧 `pose` (从 `poses[0]` 读出 $[x_0, z_0, \theta_0]$).
- 权重均匀 $w_i = 1/50$.

为什么全部放在同一点? 因为我们假设初始 `pose` 基本是准确的 (从 GPS 直接给), 不需要先搜索。如果初始 `pose` 完全未知, 应该全地图均匀撒点。

6 数据结构 2: 占据栅格地图

6.1 地图的形状

地图是一个大的 2D 矩阵，每个格子存“这里有没有障碍”的概率：

地图的成员

```
class map_t:
    def __init__(s, resolution=0.5):
        s.resolution = 0.5          # 每格子 0.5 米
        s.xmin, s.xmax = -700, 700  # 地图 x 范围 (米)
        s.zmin, s.zmax = -500, 900  # 地图 z 范围 (米)
        s.szx = 2801                 # = (xmax-xmin)/resolution + 1
        s.szz = 2801
        s.cells = np.zeros((szx, szz), dtype=np.int8)    # 0/1 二值化地图
        s.log_odds = np.zeros((szx, szz), dtype=np.float64) # 连续 log-odds
```

为什么要两张地图 (cells 和 log_odds)?

- **log_odds**: 连续的数值，用于贝叶斯累积更新（细节见 §6.2）。
- **cells**: 二值化 (0/1)，用于快速查询“这个格子到底有没有障碍”（观测步要用）。

6.2 Log-odds: 把贝叶斯更新变成加法

Log-odds 表示

令 p_i 为第 i 个格子占据的概率，定义

$$l_i = \log \frac{p_i}{1 - p_i}$$

- $l_i > 0$: 更倾向“占据” ($p_i > 0.5$)。
- $l_i < 0$: 更倾向“空” ($p_i < 0.5$)。
- $l_i = 0$: 不知道 ($p_i = 0.5$)。

为什么这样做？因为贝叶斯更新在 log-odds 域变成**简单加法**！

贝叶斯更新在 log-odds 域的漂亮形式

$$\underbrace{l_t}_{\text{新信念}} = \underbrace{l_{t-1}}_{\text{旧信念}} + \underbrace{\log \frac{p(z | \text{occ})}{p(z | \text{free})}}_{\text{这次观测的证据}}$$

每次观测，只需往 log_odds 表里加一个常数（依观测是命中还是穿过）。

本题的具体数字：

```
s.lidar_log_odds_occ = np.log(9)          # +2.197 (命中)
s.lidar_log_odds_free = np.log(1/9)       # -2.197 (穿过/空)
s.log_odds_max = 5e6                      # 饱和上下限
s.occupied_prob_thresh = 0.6
s.log_odds_thresh = np.log(0.6/0.4)      # +0.405 (二值化阈值)
```

6.3 从世界坐标到地图格子

地图是离散的（格子），要把连续 world 坐标 (x, z) 转成离散格子 (ix, iz) ：

坐标转索引

```
def grid_cell_from_xz(s, x, z):  
    ix = np.clip(np.floor((x - xmin) / resolution).astype(int), 0, szx-1)  
    iz = np.clip(np.floor((z - zmin) / resolution).astype(int), 0, szz-1)  
    return np.vstack((ix, iz))
```

为什么 `np.clip`：防止点在地图范围外导致数组越界（会闪退）。

7 预测步：粒子怎么往前走？

7.1 从 pose 差分得到”控制信号”

我们没有真正的转向/油门数据，只有每帧 pose。但可以把连续两帧之间的位姿差当作该时刻的控制：

get_control

```
def get_control(s, t):
    pose_t = s.poses[t]      # 当前帧 pose
    pose_t1 = s.poses[t-1]   # 上一帧 pose
    # 从 3x4 矩阵中提取 [x, z, yaw]
    p_t = [pose_t[0,3], pose_t[2,3], np.arctan2(-pose_t[2,0], pose_t[0,0])]
    p_t1 = [pose_t1[0,3], pose_t1[2,3], np.arctan2(-pose_t1[2,0], pose_t1[0,0])]
    return smart_minus_2d(p_t, p_t1)
```

7.2 为什么要用 smart_minus_2d? 直接做减法不行吗？

不行！这是本题最关键的几何细节之一。

直接减法是错的

假设你在 $\theta_1 = 90^\circ$ 朝北开，然后向前（北）走 10 米，到 (0,10)。

- 你的“控制”在车自己的坐标系里是“向前 10 米”，即 $[\Delta x, \Delta z] = [10, 0]$ 。
- 但 world 坐标系里看，你的 (x, z) 变化是 $[0, 10]$ （向北）。
- 如果你直接用 world 差分 $(0, 10) - (0, 0) = (0, 10)$ 当“控制”给下一帧粒子，然后下一帧粒子如果朝东，这个“控制”就会让它往北走，而不是往前走！

正确做法：**控制必须表达在车的本地坐标系里**，再由新粒子的朝向决定它在 world 里怎么走。

7.3 SE(2) 上的加法和减法

SE(2) 刚体运动群上的 $\oplus$ 和 $\ominus$

对两个位姿 $p_1 = (x_1, z_1, \theta_1)$, $p_2 = (x_2, z_2, \theta_2)$:

$$p_1 \oplus p_2 = \begin{pmatrix} x_1 + \cos \theta_1 \cdot x_2 - \sin \theta_1 \cdot z_2 \\ z_1 + \sin \theta_1 \cdot x_2 + \cos \theta_1 \cdot z_2 \\ \theta_1 + \theta_2 \end{pmatrix}$$

$$p_2 \ominus p_1 = \begin{pmatrix} \cos \theta_1 (x_2 - x_1) + \sin \theta_1 (z_2 - z_1) \\ -\sin \theta_1 (x_2 - x_1) + \cos \theta_1 (z_2 - z_1) \\ \theta_2 - \theta_1 \end{pmatrix}$$

直觉：平移项要先旋转到正确坐标系再加减。

7.4 最终 predict 步

dynamics_step

```
def dynamics_step(s, t):  
    u = s.get_control(t) # 本地 frame 的控制  
    for i in range(s.n):  
        noise = np.random.multivariate_normal(np.zeros(3), s.Q)  
        s.p[:, i] = smart_plus_2d(s.p[:, i], u + noise) # 每个粒子自己往前走
```

噪声 Q : $\text{diag}(1e-4, 1e-4, 1e-5)$ ——很小，只让粒子在真值附近小幅扰动.

8 观测步：LiDAR 怎么更新粒子权重？

8.1 核心思路：一致性评分

对每个粒子 $p^{(i)}$ ：

1. 假设这个粒子的 pose 是真的。
2. 把当前帧 LiDAR 点云按这个 pose 投到 world 坐标。
3. 去地图上查：这些点落在的格子里有多少是已知占据的？
4. 落在占据格子的点越多 $\rightarrow$ 这个假设越一致 $\rightarrow$ 权重越大。

8.2 LiDAR 投影链

lidar2world (简化版)

```
def lidar2world(s, p, points):
    # points: (N, 3) 在 velodyne frame
    # 1. velodyne -> camera (用 calibration Tr)
    pts_h = make_homogeneous(points.T) # (4, N)
    pts_cam = s.calib @ pts_h # Tr 是 (3,4), 结果 (3, N)
    # 2. camera -> world (用粒子 pose 构造的 4x4 矩阵)
    x, z, theta = p[0], p[1], p[2]
    T_world = np.array([
        [cos(theta), 0, sin(theta), x],
        [0, 1, 0, 0],
        [-sin(theta), 0, cos(theta), z],
        [0, 0, 0, 1]
    ])
    pts_world = T_world @ make_homogeneous(pts_cam)
    return pts_world[:3, :].T # (N, 3) 在 world frame
```

为什么旋转矩阵长这样？因为 θ 是绕 y 轴的 yaw（见 §5）：

$$R_y(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ -\sin \theta & 0 & \cos \theta \end{bmatrix}$$

8.3 清洗点云

原始 10K+ 点里有一堆没用的，要过滤：

clean_point_cloud

```
def clean_point_cloud(pc):
    valid_intensity = pc[:, 3] > 0.05 # 滤掉空气/噪声反射
    valid_height1 = pc[:, 2] > -1.0 # 滤掉地面
    valid_height2 = pc[:, 2] < 1.5 # 滤掉太高的（树冠/天花板）
    valid_distance = np.linalg.norm(pc[:, :2], axis=1) < 30 # 只保留 30m
    内
    return pc[valid_intensity & valid_height1 & valid_height2 &
              valid_distance]
```

为什么每个条件都必要?

- intensity: 过滤随机噪声 + 空气散射.
- 高度上下限: 只留立面障碍 (墙、车、树干); 地面和天花板对建图没用.
- 距离 < 30m: 远处 LiDAR 精度差, 且多数远点其实是前方路面延伸的噪声.

8.4 计算观测 log-likelihood

observation_step 核心

```
for i in range(s.n):
    world_pts = s.lidar2world(s.p[:, i], points)  # 用粒子 i 投影
    cells = s.map.grid_cell_from_xz(world_pts[:,0], world_pts[:,2])
    obs_logp[i] = np.sum(s.map.cells[cells[0], cells[1]])
    # map.cells 是二值化地图, 所以这是 "投到占据格子的点数"
```

直觉: 粒子 i 的 obs_logp = “在这个粒子的位姿假设下, 有多少 LiDAR 点落进了已知占据的格子”. 越大越一致.

8.5 log 域权重更新

$$\log w_i^{\text{new}} = \log w_i^{\text{old}} + \text{obs_logp}_i$$

再用 **log-sum-exp** 做归一化 (防止下溢):

$$\log w_i \leftarrow \log w_i - \underbrace{\left(a^* + \log \sum_j e^{\log w_j - a^*} \right)}_{\text{LSE, } a^* = \max_j \log w_j}$$

代码实现

```
@staticmethod
def log_sum_exp(w):
    return w.max() + np.log(np.exp(w - w.max()).sum())

@staticmethod
def update_weights(w, obs_logp):
    log_w = np.log(w) + obs_logp
    log_w -= slam_t.log_sum_exp(log_w)  # 归一化
    return np.exp(log_w)
```

为什么必须 **log-sum-exp** 而不是直接 $w \leftarrow w / \sum w$? 因为 $\sum_j w_j$ 可能下溢成 0 (w_j 本身很小) $\rightarrow$ 除 0 错误.

9 更新地图：用哪个粒子？

观测更新完权重之后，还要用新的观测**更新地图**。用哪个粒子的投影？

本题**只用权重最大的那个粒子**（简化版 FastSLAM）：

地图更新

```
best_idx = np.argmax(s.w)
best_p = s.p[:, best_idx]
world_pts = s.lidar2world(best_p, points)
occ_cells = s.map.grid_cell_from_xz(world_pts[:,0], world_pts[:,2])

# 所有 cell +free (小贡献, 相当于"这个区域都被扫过一遍了")
s.map.log_odds += s.lidar_log_odds_free
# 占据 cell 再 +occ (大贡献)
s.map.log_odds[occ_cells[0], occ_cells[1]] += s.lidar_log_odds_occ

# 饱和
np.clip(s.map.log_odds, -s.map.log_odds_max, s.map.log_odds_max, out=s.map.log_odds)

# 重新二值化
s.map.cells = (s.map.log_odds > s.map.log_odds_thresh).astype(np.int8)
```

本题的偷懒：没做射线扫描

严格的 occupancy grid 应该用 **Bresenham 射线追踪**：对每个 LiDAR 点，沿光线路径把所有穿过的 cell 标 free，终点 cell 标 occ。

本题偷懒做法：全地图每个 cell 都 +free，然后占据 cell 再 +occ 一次。

- 好处：快（向量化）。
- 坏处：未被扫描到的区域也被错误地标 free，长远看 log_odds 会偏。

好在 log_odds_max 饱和 + 大区域长期不被更新 → 实际影响不大。

10 重采样：何时、为什么

10.1 粒子退化问题

几次观测之后，会发生这样的情况：

- 少数几个粒子权重极大 (~ 0.9) .
- 其余粒子权重几乎为 0.

这叫 **degeneracy**。后果：真正代表不确定性的粒子只有 1-2 个 $\rightarrow$ Monte Carlo 估计方差大.

10.2 N_{eff} ：有效粒子数

Kish 的有效粒子数公式

$$N_{\text{eff}} = \frac{1}{\sum_i w_i^2} \in [1, N]$$

- 权重均匀 $w_i = 1/N \rightarrow N_{\text{eff}} = N$.
- 权重全集中在一个 $w_1 = 1 \rightarrow N_{\text{eff}} = 1$.

直觉：“等效于多少个均匀权重粒子”.

何时重采样

```
def resample_particles(s):
    e = 1 / np.sum(s.w**2)
    if e / s.n < s.resampling_threshold:    # 本题 0.3
        s.p, s.w = s.stratified_resampling(s.p, s.w)
```

为什么不每帧都重采样？重采样本身也会引入方差（“impoverishment”——粒子变成 clone） $\rightarrow$ 只在必要时做.

10.3 Stratified Resampling

Stratified Resampling 算法

1. 算累积权重 $W_j = \sum_{i \leq j} w_i$.
2. 把 $[0, 1]$ 均匀分成 N 段 $[\frac{i-1}{N}, \frac{i}{N}]$ ，每段采一个 $u_i \sim \mathcal{U}(\frac{i-1}{N}, \frac{i}{N})$.
3. 对每个 u_i ，找满足 $W_{j-1} < u_i \leq W_j$ 的 j ，新粒子就是 p_j .

代码

```
@staticmethod
def stratified_resampling(p, w):
    n = p.shape[1]
    cumsum = np.cumsum(w)
    u = (np.arange(n) + np.random.uniform(0, 1, n)) / n
    idx = np.searchsorted(cumsum, u)
    idx = np.clip(idx, 0, n-1)
    new_p = p[:, idx]
```



```
new_w = np.ones(n) / n          # 重采样后权重重置均匀  
return new_p, new_w
```

为什么 stratified 比 multinomial 好？保证每 $1/N$ 段至少采一次 $\rightarrow$ 方差小 $\rightarrow$ 粒子多样性更好.

11 把所有步骤连起来：主循环

run_slam

```
def run_slam(src_dir, log_dir, idx):
    slam = slam_t(resolution=0.5, Q=np.diag([1e-4, 1e-4, 1e-5]))
    slam.read_data(src_dir, idx)

    # 从第一帧 pose 提取初始 [x, z, yaw]
    p0 = [d[0,0,3], d[0,2,3], arctan2(-d[0,2,0], d[0,0,0])]

    # 用 1 个粒子初始化地图
    slam.init_particles(n=1, p=p0.reshape(3,1), w=[1.0])
    slam.observation_step(t=0)          # 第一帧用来建初始地图

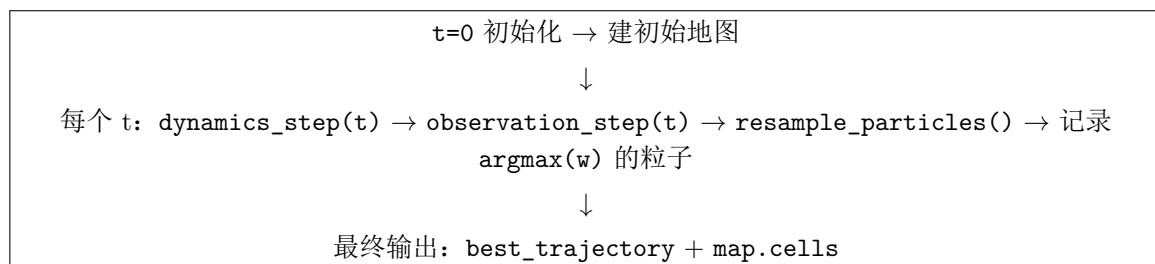
    # 真正的粒子集：50 个，都放在 p0
    slam.init_particles(n=50, p=np.tile(p0, (1, 50)), w=np.ones(50)/50)

    best_trajectory = [p0]
    for t in range(1, T):
        slam.dynamics_step(t)           # 所有粒子按控制 + 噪声前进
        slam.observation_step(t)        # 用 LiDAR 更新权重 + 地图
        slam.resample_particles()        # 若 N_eff 低则重采样

        best_idx = np.argmax(slam.w)
        best_trajectory.append(slam.p[:, best_idx].copy())

    return np.array(best_trajectory)
```

11.1 流程总结图



12 “毛刺”问题——轨迹为什么不光滑

12.1 现象

渲染 `best_trajectory` 出来你会发现：轨迹不是一条光滑的曲线，而是一路抖动，像被揉皱了一样。

12.2 6 个根本原因

毛刺的 6 个原因（按影响从大到小）

原因 1 · `argmax` 粒子切换（最主要）

每一帧选 `argmax(w)` 作为“最佳粒子”的 ID。不同帧的最佳粒子可能是不同的粒子！而不同粒子的位置本来就有差（粒子噪声），所以轨迹点从粒子 A 跳到粒子 B 时会有离散跳。

原因 2 · 观测似然非光滑

$\text{obs_logp} = \sum \text{map_binary}$ ，是个整数。可能很多粒子得到相同 `obs_logp`（都投到了同样数量的占据格），`argmax` 会在这些 tie 的粒子之间随机切换。

原因 3 · Resampling 的离散化

重采样把低权重粒子杀掉，复制高权重粒子 → 粒子都变成几个位置的 clone。下一轮再加噪声后，粒子位置重新抖散 → `argmax` 又跳。

原因 4 · 动力学噪声

每步给粒子加 $\mathcal{N}(0, Q)$ 噪声（ $\sigma \sim 10^{-2}$ ），即便很小，也会在 `argmax` 粒子身上体现为抖动。

原因 5 · 有限 Monte Carlo 误差

$N = 50$ 对 $\mathbb{R}^3$ 后验其实挺稀疏，本身就有 $O(1/\sqrt{N}) \approx 14\%$ 的采样误差。

原因 6 · 地图反馈回路

地图用 `argmax` 粒子更新 → 如果某帧 `argmax` 选错了，地图就有了轻微错误 → 影响下一帧所有粒子的观测 likelihood → 间接放大抖动。

12.3 四种缓解方法

1. **加权平均代替 `argmax`**: $\text{best_p} = \sum_i w_i p_i$ 。一行代码的修改，可以消掉原因 1 和 2。
2. **平滑似然**: 把 `sum(map_binary)` 改成 `sum(map.log_odds)` 或者加个高斯核 → likelihood 连续 → 粒子 tie 变少。
3. **增加粒子数**: $N = 200$ 或 500，减小 Monte Carlo 误差。
4. **真正的 FastSLAM**: 每个粒子自己维护一张地图（内存代价 $\times N$ ）→ 地图误差不会反馈到 likelihood。

为什么本题没用这些缓解方法？

因为作业就是要暴露这些典型的 PF SLAM 问题。教授问这题时想看你是否：

1. 说得出这 6 个原因里的至少 3 个。
2. 区分“degeneracy”（resample 前）vs “impoverishment”（resample 后）。
3. 能主动说出改进方案并指出 trade-off（内存 vs 精度等）。

13 验证：怎么判断 SLAM 跑对了？

1. **轨迹 vs odom 对比图**：PF 轨迹应大致沿 odom 走，但在某些地方应偏离 odom 且更合理（这说明 LiDAR 观测起了作用）。
2. **地图质量**：
 - 墙面连续清晰（不是两层或糊掉）。
 - 转角分明。
 - 空的道路显示为白色/浅色（正确标 free）。
 - 无 “ghost obstacles”（某些地方出现不存在的墙）。
3. **N_{eff} 动态**：
 - 大部分时间 $N_{\text{eff}} \approx N$ （粒子权重均匀）。
 - 偶尔跌落触发 resample，之后恢复。
 - 若持续很低 → 问题（粒子普遍和观测不一致）。
4. **毛刺**：接受存在（见 §11），但肉眼整体趋势应清晰。

14 口试速查 · 常见陷阱

Trap 1 · “这是真正的 FastSLAM 吗？”

不是。本题是简化版：所有粒子共享一张地图，用 argmax 粒子更新地图。

真 FastSLAM：每粒子自带一张独立地图（通过 Rao-Blackwellization 把地图 analytically marginalize 掉）。代价是内存 $\times N$ 。

Trap 2 · “为什么用 2D 状态不是 3D？”

因为 KITTI 车在路面上开， y 方向（垂直）几乎不变。降到 $[x, z, \theta]$ 后：

- 搜索空间从 $\mathbb{R}^6$ 降到 $\mathbb{R}^3$ 。
- 粒子数需求降（粒子滤波对维度敏感）。
- 地图也是 2D，省内存。

Trap 3 · “为什么观测模型是非线性/不可微的？”

因为观测步要做查表：“把 LiDAR 点投到地图，查这些格子占据吗”。这是个 indexing 操作，对 pose 不可微（pose 微小变化 $\rightarrow$ 点跨过 cell 边界 $\rightarrow$ 离散跳变）。

EKF 需要 Jacobian $\rightarrow$ 不行；PF 只要能算 likelihood $\rightarrow$ 行。

Trap 4 · “为什么初始化 50 个粒子都放一个点？方差不就是 0 了？”

对，初始化时方差为 0。但第一步 dynamics_step 加了 noise 之后它们就分散开了。设计意图是：我们相信初始 pose 很准，不需要先广搜。

反之，若初始 pose 完全未知，应该均匀撒满全地图，让 PF 靠观测收敛。

Trap 5 · “log-odds 为什么不会一直涨？”

代码有饱和：`np.clip(log_odds, -5e6, +5e6)`。否则无限加下去会 overflow。

但实际上 `log_odds_max = 5 × 106` 非常大，正常运行中很少到达。

Trap 6 · “cells 独立假设什么时候失效？”

大物体（汽车、长墙）跨多个 cell，真实世界里这些 cell 高度相关（要么都是墙，要么都不是）。

log-odds 更新假设独立 $\rightarrow$ 信息被重复计算 $\rightarrow$ 地图“过度自信”。

实际影响：地图看起来“太肯定”，哪怕只扫过一次就直接变黑。

15 考题：按学习路径排序

数据与问题理解（必会）：

1. KITTI 给你哪些数据？每种数据的 shape 和物理含义？
2. 什么是 SLAM？为什么是“鸡生蛋”问题？
3. 为什么只用 odometry 不行？只用 LiDAR 不行？
4. 为什么状态只用 2D $[x, z, \theta]$ 而不是 6D？

粒子滤波基础：

5. 粒子滤波和 KF/EKF 的本质区别？什么场景下 PF 必要？
6. PF 的三大动作是什么？本题各自怎么实现？
7. 推导 $N_{\text{eff}} = 1 / \sum w_i^2$ (Kish 公式)。
8. 为什么每帧不都 resample？
9. Stratified vs multinomial resampling 的方差差异？

工程细节（中级）：

10. 推导 smart_plus_2d 公式；为什么平移要乘 $R(\theta)$ ？
11. Log-odds 表示好在哪？（加法更新 + 数值稳定 + 独立假设下易算）
12. LiDAR 投影链是什么？每一步的矩阵代表什么？
13. 为什么要 log-sum-exp 归一化而不是直接除 $\sum w$ ？
14. 点云 clean 的 3 个条件各防什么？

失败模式（高级）：

15. 为什么轨迹有毛刺？至少给出 3 个原因。
16. “Degeneracy” 和 “impoverishment” 的区别？
17. 本题是否真正的 FastSLAM？差别在哪？
18. 本题 cells 独立假设什么时候失效？影响是什么？
19. 如果初始 pose 完全未知，这个 PF 还能工作吗？

改进方案（加分）：

20. 如何消除轨迹毛刺？至少 2 种方法。
21. 如何把这个简化版改成真正的 FastSLAM？代价是什么？
22. 如果 $N = 50$ 不够，怎么判断？怎么调？
23. 本题为什么没实现 Bresenham 射线追踪？影响是什么？

“PF 是粒子逼近后验，每个粒子是一个假设；
对不起它也有毛刺，但毛刺原因你要说得出来。”